# Task 5: Set Up Monitoring Using Prometheus & Grafana

*Full-stack telemetry collection, scraping configuration, and real-time dashboard visualization*

| Field | Value |
|---|---|
| Course Activity | Lab 3: Monitoring, Logging & DevSecOps |
| Student Name | Garvit Sharma |
| Roll / Register No. | 2301730190 |
| Email Address | garvitsharma8909@gmail.com |
| Execution Date | November 04, 2026 |

## 1. Objective & Problem Statement

Set up monitoring using Prometheus and visualize metrics using Grafana dashboards for a target application or system under synthetic workload.

## 2. Telemetry Architecture & Metrics

The observability pipeline leverages Prometheus pull scraping and Grafana dashboard provisioning:

• **Instrumented Metrics (/metrics):**

- demo_http_requests_total: Counter tracking requests partitioned by method, path, status.

- demo_http_request_duration_seconds: Histogram measuring request latency distributions.

- demo_feature_greeting_v2_enabled: Gauge reflecting dynamic runtime feature flag state.

• **Prometheus Scraper:** Configured with 15-second interval querying targets 'prometheus:9090' and 'demo-app:8080'.

• **Grafana Auto-Provisioning:** Automatically provisions Prometheus datasource and loads the 'Demo Application Overview' dashboard upon startup.

## 3. Execution Commands

```
docker compose up -d demo-app prometheus grafana
Invoke-WebRequest http://localhost:8080/api/status
locust -f tests/load/locustfile.py --headless -u 10 -r 2 -t 30s --host http://localhost:8080
```

## 4. Photographic Evidence & Dashboard Telemetry




*Figure: Grafana Demo Application Overview dashboard showing request throughput and latency*

## 5. Observations & Key Findings

• **Real-Time Visibility:** Synthetic traffic generated by Locust was captured immediately in Grafana, reflecting a throughput of ~8.64 req/s and median latency of 7.1ms.

• **Granular Route Telemetry:** Partitioning metrics by route path allows instant isolation of high-traffic endpoints versus background API calls.